

```
import pandas as pd
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

from sklearn.pipeline import make_pipeline

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

```
uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(
    filename,
    sep='\\t', # Specify literal backslash-t as the separator
    header=None # Indicate that there is no header row
)

print("File loaded:", filename)
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())

display(df.head())
```

Choose Files SMSSpamCollection.csv

SMSSpamCollection.csv(text/csv) - 489217 bytes, last modified: 8/17/2026 - 100% done

Saving SMSSpamCollection.csv to SMSSpamCollection (5).csv

File loaded: SMSSpamCollection (5).csv

Dataset shape: (5574, 2)

Columns: [0, 1]

/tmp/ipykernel_746/3507169897.py:5: ParserWarning: Falling back to the 'python' engine because the 'c' engine does not support regex separators (separators > 1 char and different from '\s+':
df = pd.read_csv(

	0	1
0	"ham	Go until jurong point, crazy.. Available only ...
1	"ham	Ok lar... Joking wif u oni..."
2	"spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	"ham	U dun say so early hor... U c already then say...
4	"ham	Nah I don't think he goes to usf, he lives aro...

◆ Gemini

```
# Standardize column names
df.columns = [
    str(col).strip().lower()
    for col in df.columns
]

# Detect label column
if "label" in df.columns:
    label_col = "label"
elif "v1" in df.columns:
    label_col = "v1"
elif "category" in df.columns:
    label_col = "category"
elif "class" in df.columns:
    label_col = "class"
elif "0" in df.columns:
    label_col = "0"
else:
    raise ValueError(
        "Could not find the spam/ham label column."
    )

# Detect message column
if "message" in df.columns:
    message_col = "message"
elif "v2" in df.columns:
    message_col = "v2"
elif "text" in df.columns:
    message_col = "text"
elif "sms" in df.columns:
    message_col = "sms"
elif "1" in df.columns:
    message_col = "1"
else:
    raise ValueError(
```

```

    "Could not find the SMS/text message column."
)

# Keep only required columns
df = df[
    [label_col, message_col]
].copy()

df.columns = [
    "label",
    "message"
]

# Remove leading double quotes and strip spaces from 'label' column
df["label"] = df["label"].str.strip('\"').str.strip()
# Strip spaces from 'message' column
df["message"] = df["message"].str.strip()

# Remove missing values
df = df.dropna(
    subset=["label", "message"]
)

# Convert text to string
df["message"] = df["message"].astype(str)

X = df["message"]
y = df["label"]

print("Dataset prepared successfully!")
print("Number of records:", len(df))
print("\nClasses:")
print(y.value_counts())

display(df.head())

```

Dataset prepared successfully!
Number of records: 5574

Classes:

label

ham 4827

spam 747

Name: count, dtype: int64

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni..."
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

nb_model = make_pipeline(
    TfidfVectorizer(),
    MultinomialNB()
)

nb_model.fit(
    X_train,
    y_train
)

nb_predictions = nb_model.predict(
    X_test
)

nb_accuracy = accuracy_score(
    y_test,
    nb_predictions
)

print(
    "Naïve Bayes Accuracy:",
    round(nb_accuracy * 100, 2),
    "%"
)
```

Naïve Bayes Accuracy: 95.98 %

```
lr_model = make_pipeline(
    TfidfVectorizer(),
    LogisticRegression(
        max_iter=1000
    )
)

lr_model.fit(
    X_train,
    y_train
)

lr_predictions = lr_model.predict(
    X_test
)

lr_accuracy = accuracy_score(
    y_test,
    lr_predictions
)
```

```
print(
    "Logistic Regression Accuracy:",
    round(lr_accuracy * 100, 2),
    "%"
)
```

Logistic Regression Accuracy: 97.27 %

```
models = [
    "Naïve Bayes",
    "Logistic Regression"
]

accuracies = [
    nb_accuracy,
    lr_accuracy
]

plt.figure(figsize=(8, 5))

plt.bar(
    models,
    accuracies
)

plt.ylabel("Accuracy")
plt.xlabel("Model")

plt.title(
    "Naïve Bayes vs Logistic Regression"
)

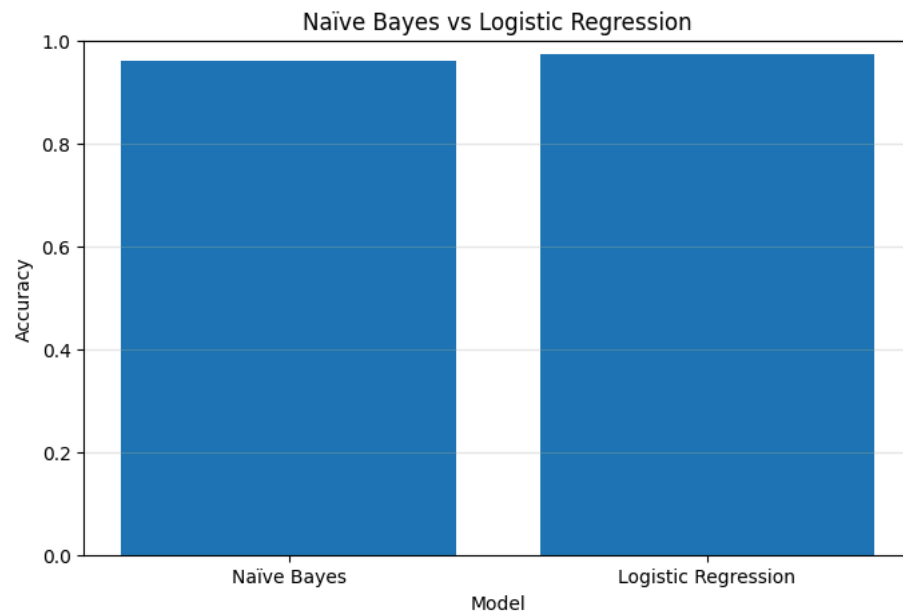
plt.ylim(0, 1)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()

print("Naïve Bayes:",
      round(nb_accuracy * 100, 2), "%")

print("Logistic Regression:",
      round(lr_accuracy * 100, 2), "%")
```



Naïve Bayes: 95.98 %
Logistic Regression: 97.27 %

```
fig, axes = plt.subplots(
    1,
    2,
    figsize=(12, 5)
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    nb_predictions,
    ax=axes[0]
)

axes[0].set_title(
    "Naïve Bayes Confusion Matrix"
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    lr_predictions,
    ax=axes[1]
)

axes[1].set_title(
    "Logistic Regression Confusion Matrix"
)

plt.tight_layout()
plt.show()
```

```

print("===== FINAL RESULT =====")

if nb_accuracy > lr_accuracy:
    print("Naïve Bayes performed better.")
elif lr_accuracy > nb_accuracy:
    print("Logistic Regression performed better.")
else:
    print("Both models achieved the same accuracy.")

print()
print(
    "Naïve Bayes:",
    round(nb_accuracy * 100, 2),
    "%"
)

print(
    "Logistic Regression:",
    round(lr_accuracy * 100, 2),
    "%"
)

```

